EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 4 | Semana 8

Integração e revisão

INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026

Apresentação

Esta apostila reúne os conceitos centrais estudados nas Semanas 1 a 7 da disciplina. A proposta é revisar como hardware, software e dados se relacionam durante a execução de um programa. Use as seções para retomar definições, acompanhar exemplos e identificar os pontos que precisam de nova leitura.

O que você deverá conseguir fazer

- relacionar os principais componentes do computador e explicar suas funções;
- interpretar representações binárias e reconhecer carry e overflow em exemplos de largura fixa;
- descrever como o processador busca, decodifica e executa instruções;
- distinguir registradores, cache, memória RAM e armazenamento e explicar o papel de entrada e saída;
- comparar tendências associadas a RISC e CISC e explicar pipeline, vazão e riscos;
- integrar esses conceitos ao analisar a execução de um programa simples.

Como aproveitar a revisão

1. Leia cada seção tentando explicar o conceito com suas próprias palavras.
2. Refaça as conversões e operações binárias sem consultar o resultado.
3. Ao estudar o exemplo integrado, acompanhe o caminho do programa, dos dados e das instruções.
4. Responda às questões de autoavaliação e confira as respostas comentadas ao final.
5. Retome as apostilas e videoaulas das semanas anteriores nos tópicos em que ainda houver dúvida.

1. Componentes e funções do computador

Um sistema computacional combina componentes físicos e programas. O hardware oferece recursos para processar, armazenar, receber e apresentar informações; o software descreve as operações que esses recursos devem realizar. A comunicação entre componentes ocorre por caminhos de dados, sinais de controle e interfaces de entrada e saída.

Componente	Função principal	Exemplo de relação com um programa
Processador (CPU)	Interpreta instruções e coordena operações.	Busca instruções e realiza operações aritméticas e lógicas.
Memória principal (RAM)	Mantém temporariamente programas e dados em uso.	Armazena partes do programa carregado pelo sistema operacional.
Armazenamento	Preserva dados sem energia.	Mantém executáveis, documentos e arquivos em SSD ou HDD.
Entrada e saída	Permite a troca de informações com o ambiente.	Recebe comandos do teclado e apresenta resultados na tela.
Barramentos e interconexões	Transportam dados, endereços e sinais de controle.	Permitem a comunicação entre CPU, memória e controladores.

Dentro da CPU, a Unidade Lógica e Aritmética (ULA) executa operações como soma, comparação e operações lógicas. A unidade de controle interpreta a instrução e organiza os sinais necessários. Os registradores guardam valores pequenos e imediatos, como operandos, endereços e resultados parciais.

2. Bits, bases numéricas e operações

O bit é a menor unidade de informação representada por 0 ou 1. Um byte corresponde a oito bits. Como circuitos digitais trabalham com estados binários, números e instruções são codificados em padrões de bits. Decimal usa dez símbolos, binário usa dois e hexadecimal usa dezesseis.

Exemplo 1. $37_{10} = 100101_2 = 25_{16}$. O hexadecimal $3A_{16}$ vale $3 \times 16 + 10 = 58_{10}$, e pode ser escrito em binário como 00111010_2.

Em operações com largura fixa, o número de bits disponível limita a faixa representável. Para n bits sem sinal, a faixa é de 0 a $2^n - 1$. Em complemento de dois com sinal, a faixa é de -2^{n-1} a $2^{n-1} - 1$. O complemento de dois forma números negativos invertendo os bits do valor positivo e somando 1.

Exemplo 2. Em 4 bits, +3 é 0011. Para representar −3, invertemos para 1100 e somamos 1: 1101.

Carry e overflow

Carry indica que uma soma sem sinal produziu um bit além da largura disponível. Overflow indica que o resultado de uma operação com sinal ficou fora da faixa representável. São condições distintas.

Operação em 4 bits	Interpretação	Sinalização
1111 + 0001 = 1 0000	15 + 1 em aritmética sem sinal; os quatro bits armazenados são 0000.	Carry-out = 1.
0111 + 0001 = 1000	+7 + +1 em complemento de dois; +8 não cabe na faixa de −8 a +7.	Overflow = 1; carry-out = 0.

3. Processador e ciclo de instrução

Um programa é executado por uma sequência de instruções. Em um modelo introdutório, o processador busca uma instrução na memória, decodifica seu código de operação e identifica os operandos, executa a tarefa e registra o resultado. O contador de programa (PC) indica a próxima instrução; o registrador de instrução (IR) mantém a instrução em análise.

Etapa do ciclo	Ação resumida
Busca (fetch)	O endereço indicado pelo PC é usado para obter a próxima instrução; o PC é atualizado conforme o fluxo previsto.
Decodificação (decode)	A unidade de controle interpreta a instrução e determina quais recursos e operandos serão usados.
Execução (execute)	A ULA ou outra unidade realiza a operação; uma instrução de acesso à memória pode calcular um endereço.
Acesso à memória	Quando necessário, o processador lê ou escreve um dado na memória.
Escrita do resultado	O resultado é gravado em um registrador ou destino definido pela instrução.

Uma instrução de máquina inclui um código de operação e pode indicar um ou mais operandos. O modo de endereçamento determina como localizar um valor. No modo imediato, o próprio valor aparece na instrução; no modo por registrador, o operando está em um registrador; no modo direto, a instrução informa um endereço de memória; no indireto, um registrador ou posição contém o endereço do dado.

Leitura do exemplo. Em uma instrução hipotética “somar R1, R2”, os operandos estão em registradores. Em “somar R1, #5”, o valor 5 é imediato. A sintaxe concreta depende da ISA.

4. Memória, cache e entrada e saída

O computador combina memórias com capacidades, velocidades e custos diferentes. Em geral, os níveis mais próximos da CPU são menores e rápidos; os níveis mais distantes armazenam mais dados e preservam arquivos por mais tempo.

Nível	Proximidade da CPU	Capacidade relativa	Volatilidade
Registradores	Dentro da CPU	Muito pequena	Volátil
Cache	Próxima da CPU	Pequena	Volátil
RAM	Conectada à CPU	Maior	Volátil
SSD ou HDD	Armazenamento secundário	Grande	Não volátil

A cache mantém cópias de blocos de memória que podem ser usados novamente. A localidade temporal é a tendência de reutilizar algo acessado há pouco; a localidade espacial é a tendência de acessar posições próximas. Quando o dado está no nível consultado ocorre hit (acerto); quando não está, ocorre miss (falta) e o sistema procura em outro nível. Uma falta não é um erro do programa.

Exemplos de localidade. Percorrer um vetor elemento a elemento tende a aproveitar localidade espacial. Atualizar repetidamente um acumulador tende a aproveitar localidade temporal.

Como a CPU interage com periféricos

Mecanismo	Como funciona	Uso típico da CPU
Polling	A CPU consulta repetidamente o estado do dispositivo.	Permanece verificando se o dispositivo está pronto.
Interrupção	O dispositivo sinaliza quando requer atenção ou conclui uma operação.	Pode executar outras instruções entre eventos.
DMA	Um controlador transfere blocos entre dispositivo e RAM após configuração.	Configura a operação e trata a conclusão; não copia cada unidade do bloco.

5. ISA, RISC e CISC

A arquitetura do conjunto de instruções (ISA) define a interface que o software compilado utiliza: instruções, registradores visíveis, formatos e regras de execução. A microarquitetura descreve como um processador concreto implementa essa interface.

Tendência associada a RISC	Tendência associada a CISC
Formatos mais regulares e operações individuais mais simples.	Maior variedade de instruções e de modos de endereçamento.
É comum separar operações aritméticas de acessos à memória.	Algumas instruções podem descrever operações compostas.
O compilador pode combinar várias instruções para expressar uma tarefa.	A implementação pode decodificar uma instrução em operações internas menores.

RISC e CISC são categorias úteis para discutir tendências de projeto; não são previsões diretas de velocidade. Uma mesma ISA pode ser implementada por microarquiteturas distintas. O desempenho depende da implementação, do compilador, do programa, dos dados e das condições de execução.

6. Pipeline e riscos

O pipeline divide a execução em etapas que podem ser ocupadas por instruções diferentes ao mesmo tempo. Um modelo didático comum utiliza IF (busca), ID (decodificação), EX (execução ou cálculo de endereço), MEM (acesso a dados) e WB (escrita do resultado). Processadores reais podem usar outras divisões.

Ciclo	IF	ID	EX	MEM	WB
1	I1	-	-	-	-
2	I2	I1	-	-	-
3	I3	I2	I1	-	-
4	-	I3	I2	I1	-
5	-	-	I3	I2	I1
6	-	-	-	I3	I2
7	-	-	-	-	I3

No exemplo ideal, três instruções independentes percorrem cinco etapas em sete ciclos. Sem sobreposição, o modelo simplificado levaria quinze ciclos. O pipeline aumenta a vazão potencial - instruções concluídas por intervalo - mas não reduz necessariamente a latência de uma instrução individual. A conta ideal pressupõe etapas equilibradas e nenhuma pausa.

Risco	Situação	Resposta possível
Estrutural	Duas etapas precisam do mesmo recurso simultaneamente.	Separar recursos ou inserir espera.
De dados	Uma instrução precisa do resultado de outra que ainda não está pronto.	Encaminhar o valor ou aguardar.
De controle	Um desvio altera o caminho das próximas instruções.	Esperar ou prever o caminho; corrigir se a previsão falhar.

Faltas de cache também podem atrasar instruções que aguardam dados. Dependências, desvios e esperas reduzem a vazão em relação ao cenário ideal; o efeito exato depende da microarquitetura.

7. Um exemplo integrado

Considere um programa que calcula a soma dos elementos de um vetor guardado em um arquivo. O percurso abaixo conecta os principais assuntos da disciplina.

1. O arquivo e o executável permanecem no SSD até serem necessários.
2. O sistema operacional carrega para a RAM as partes do programa e dos dados que serão usadas.
3. O processador busca instruções da ISA pela hierarquia de memória; blocos recentes podem estar na cache.
4. A unidade de controle decodifica instruções como carregar um valor, somá-lo ao acumulador e atualizar o índice.
5. A ULA realiza a soma; registradores mantêm o acumulador, o índice e valores temporários.
6. O pipeline pode sobrepor instruções independentes. Uma instrução que depende do novo acumulador pode precisar aguardar ou receber o resultado encaminhado.
7. Ao terminar, o programa solicita que o resultado seja apresentado; o sistema e os dispositivos de saída completam a comunicação com o usuário.

O exemplo também mostra por que não basta estudar cada componente isoladamente: o desempenho final depende do fluxo de instruções, do padrão de acesso aos dados, da memória, da implementação do processador e das operações de entrada e saída.

8. Checklist de revisão

- Consigo diferenciar hardware, software, processador, memória e entrada e saída?
- Consigo converter valores simples entre decimal, binário e hexadecimal?
- Consigo explicar bit, byte, complemento de dois, carry e overflow?
- Consigo descrever o ciclo de busca, decodificação e execução?
- Consigo identificar operandos imediatos, em registrador e na memória?
- Consigo ordenar registradores, cache, RAM e armazenamento?
- Consigo explicar localidade, hit, miss, polling, interrupção e DMA?
- Consigo diferenciar ISA de microarquitetura e falar de tendências RISC/CISC sem prever velocidade pelo rótulo?
- Consigo explicar as cinco etapas do pipeline, latência, vazão e os três tipos de risco?

9. Questões de autoavaliação

Responda primeiro sem consultar as respostas comentadas. Use-as depois para conferir seu raciocínio.

1. Organize registradores, cache, RAM e SSD do nível mais próximo da CPU ao mais distante.
2. Converta 45_{10} para binário e hexadecimal.
3. Em 4 bits sem sinal, calcule 1111 + 0001. Qual resultado cabe nos quatro bits e qual condição é indicada?
4. Represente −5 em complemento de dois usando 8 bits.
5. Dê um exemplo de localidade temporal e outro de localidade espacial em um programa.
6. Explique a diferença entre ISA e microarquitetura.
7. Classifique o operando destacado: somar R1, #5. O valor 5 está em qual modo de endereçamento?
8. Em que situação uma interrupção pode reduzir o trabalho de consulta repetida feito pela CPU? O que o DMA transfere?
9. No modelo ideal de cinco etapas, quantos ciclos são necessários para três instruções independentes? Compare com a execução sequencial simplificada.
10. A instrução I2 usa o valor calculado por I1. Que tipo de risco pode ocorrer e cite uma resposta possível do processador.

Respostas comentadas

1. Registradores → cache → RAM → SSD. A capacidade tende a crescer e a proximidade da CPU diminui ao longo dessa sequência.
2. $45_{10} = 101101_2 = 2D_{16}$. Uma conferência é 32 + 8 + 4 + 1 = 45; em hexadecimal, 2 × 16 + 13 = 45.
3. $1111_2 + 0001_2 = 1\ 0000_2$. Os quatro bits guardam 0000 e há carry-out igual a 1.
4. +5 em 8 bits é 00000101. Inverter e somar 1 resulta em 11111011, representação de −5.
5. Temporal: reutilizar um acumulador em um laço. Espacial: percorrer posições consecutivas de um vetor.
6. ISA define a interface de instruções visível ao software; microarquitetura descreve a implementação interna em um processador específico.
7. Imediato: o valor 5 está codificado na própria instrução (a notação # é apenas ilustrativa).
8. A interrupção permite que o processador faça outro trabalho até o dispositivo sinalizar um evento. DMA transfere blocos entre um dispositivo e a RAM após ser configurado.
9. Sete ciclos ideais. Executando cada instrução separadamente, cinco etapas por instrução dariam quinze ciclos no modelo simplificado.
10. Pode ocorrer risco de dados. O processador pode encaminhar o resultado diretamente ou inserir ciclos de espera.